

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 1**



ANDROID BASIC WITH KOTLIN

Oleh:

Muhammad Dzul Fathi Ahyan

NIM. 2410817210011

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MARET 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN I
MODUL 1

Laporan Praktikum Pemrograman Mobile Modul 1: Android Basic with Kotlin ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Dzul Fathi Ahyan

NIM : 2410817210011

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Raymond Hariyono
NIM. 2310817210007

Erika Maulidiya, S.Kom.,M.Kom
NIP. 200006192025062016

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1.....	6
A. Source Code	9
Source Code XML	9
Source Code Compose	11
B. Output Program.....	14
C. Pembahasan.....	16
Pembahasan XML	16
Pembahasan Compose.....	21
SOAL 2.....	26
A. Pembahasan.....	26
SOAL 3.....	28
A. Pembahasan.....	28
DAFTAR PUSTAKA.....	30
TAUTAN GIT	31

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1 XML	14
Gambar 2. Screenshot Hasil Jawaban Soal 1 XML	14
Gambar 3. Screenshot Hasil Jawaban Soal 1 Compose	15
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose	15

DAFTAR TABEL

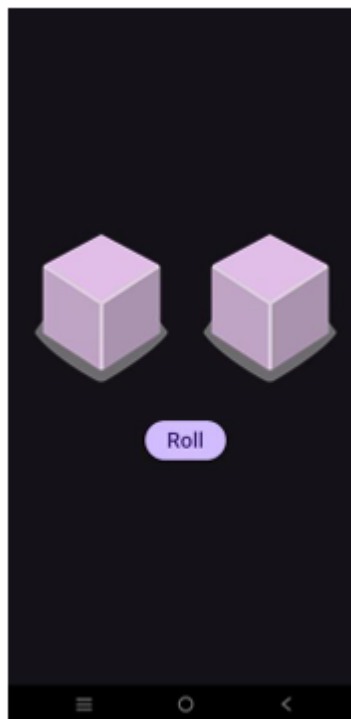
Tabel 1. Source Code Jawaban Soal 1 XML.....	9
Tabel 2. Source Code Jawaban Soal 1 XML.....	10
Tabel 3. Source Code Jawaban Soal 1 Compose.....	11

SOAL 1

Soal Praktikum:

Buatlah sebuah aplikasi yang dapat menampilkan 2 buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



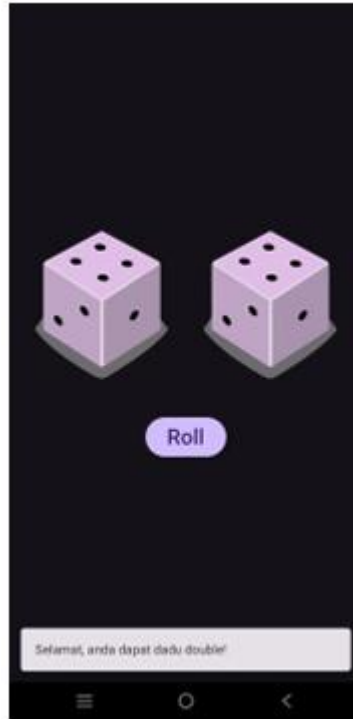
Gambar 1 Tampilan Awal Aplikasi

2. Setelah user menekan tombol “Roll Dice” maka masing-masing dadu akan memunculkan sisi dadu masing-masing dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2 maka akan menampilkan pesan “Anda belum beruntung!” seperti dapat dilihat pada Gambar 2.



Gambar 2 Tampilan Dadu Setelah Di Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat anda dapat dadu double” seperti dapat dilihat pada Gambar 3.



Gambar 3 Tampilan Roll Dadu Double

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.
5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam folder Modul 1 dalam bentuk Project. Jangan lupa untuk melakukan Clean Project sebelum mengupload pekerjaan anda pada repository.
6. Untuk gambar dadu dapat didownload pada link berikut:
https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp=sharing

A. Source Code

Source Code XML

MainActivity.kt

Tabel 1. Source Code Jawaban Soal 1 XML

1.	package com.example.modul1xml
2.	
3.	import android.os.Bundle
4.	import android.widget.Button
5.	import android.widget.ImageView
6.	import android.widget.Toast
7.	import androidx.appcompat.app.AppCompatActivity
8.	
9.	class MainActivity : AppCompatActivity() {
10.	
11.	override fun onCreate(savedInstanceState: Bundle?) {
12.	super.onCreate(savedInstanceState)
13.	setContentView(R.layout.activity_main)
14.	
15.	val ivDice1 =
16.	findViewById<ImageView>(R.id.ivDice1)
17.	val ivDice2 =
18.	findViewById<ImageView>(R.id.ivDice2)
19.	val btnRoll = findViewById<Button>(R.id.btnRoll)
20.	
21.	btnRoll.setOnClickListener {
22.	val dice1 = (1..6).random()
23.	val dice2 = (1..6).random()
24.	
25.	ivDice1.setImageResource(getDiceImage(dice1))
26.	
27.	ivDice2.setImageResource(getDiceImage(dice2))
28.	
29.	if (dice1 == dice2) {
30.	Toast.makeText(this, "Yey dapat dobel",
31.	Toast.LENGTH_SHORT).show()
32.	} else {
33.	Toast.makeText(this, "Kada Hoki",
34.	Toast.LENGTH_SHORT).show()
	}
	}
	}
	private fun getDiceImage(diceValue: Int): Int {

```

35.         return when (diceValue) {
36.             1 -> R.drawable.dice_1
37.             2 -> R.drawable.dice_2
38.             3 -> R.drawable.dice_3
39.             4 -> R.drawable.dice_4
40.             5 -> R.drawable.dice_5
41.             6 -> R.drawable.dice_6
42.             else -> R.drawable.dice_0
43.         }
44.     }
45. }

```

activity_main.xml

Tabel 2. Source Code Jawaban Soal 1 XML

```

1. <?xml version="1.0" encoding="utf-8"?>
2. <LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/andro
   id"
3.     android:layout_width="match_parent"
4.     android:layout_height="match_parent"
5.     android:gravity="center"
6.     android:orientation="vertical"
7.     android:background="#FFFFFF">
8.
9.     <LinearLayout
10.         android:layout_width="wrap_content"
11.         android:layout_height="wrap_content"
12.         android:orientation="horizontal"
13.         android:layout_marginBottom="32dp">
14.
15.         <ImageView
16.             android:id="@+id/ivDice1"
17.             android:layout_width="120dp"
18.             android:layout_height="120dp"
19.             android:layout_marginEnd="16dp"
20.             android:src="@drawable/dice_0"
21.             android:contentDescription="Dadu 1" />
22.
23.         <ImageView
24.             android:id="@+id/ivDice2"
25.             android:layout_width="120dp"
26.             android:layout_height="120dp"
27.             android:layout_marginStart="16dp"
28.             android:src="@drawable/dice_0"
29.             android:contentDescription="Dadu 2" />

```

30.	</LinearLayout>
31.	
32.	<Button
33.	android:id="@+id/btnRoll"
34.	android:layout_width="wrap_content"
35.	android:layout_height="wrap_content"
36.	android:text="Roll" />
37.	
38.	</LinearLayout>

Source Code Compose

MainActivity.kt

Tabel 3. Source Code Jawaban Soal 1 Compose

1.	package com.example.modullcompose
2.	
3.	import android.os.Bundle
4.	import android.widget.Toast
5.	import androidx.activity.ComponentActivity
6.	import androidx.activity.compose.setContent
7.	import androidx.compose.foundation.Image
8.	import androidx.compose.foundation.layout.Arrangement
9.	import androidx.compose.foundation.layout.Column
10.	import androidx.compose.foundation.layout.Row
11.	import androidx.compose.foundation.layout.Spacer
12.	import androidx.compose.foundation.layout.fillMaxSize
13.	import androidx.compose.foundation.layout.height
14.	import androidx.compose.foundation.layout.size
15.	import androidx.compose.material3.Button
16.	import androidx.compose.material3.MaterialTheme
17.	import androidx.compose.material3.Surface
18.	import androidx.compose.material3.Text
19.	import androidx.compose.runtime.Composable
20.	import androidx.compose.runtime.getValue
21.	import androidx.compose.runtime.mutableStateOf
22.	import androidx.compose.runtime.remember
23.	import androidx.compose.runtime.setValue
24.	import androidx.compose.ui.Alignment
25.	import androidx.compose.ui.Modifier
26.	import androidx.compose.ui.platform.LocalContext
27.	import androidx.compose.ui.res.painterResource
28.	import androidx.compose.ui.unit.dp
29.	
30.	class MainActivity : ComponentActivity() {
31.	override fun onCreate(savedInstanceState: Bundle?) {

```

32.         super.onCreate(savedInstanceState)
33.         setContent {
34.             MaterialTheme {
35.                 Surface(
36.                     modifier = Modifier.fillMaxSize(),
37.                     color = MaterialTheme.colorScheme.background
38.                 ) {
39.                     DiceRollerApp()
40.                 }
41.             }
42.         }
43.     }
44. }
45.
46. @Composable
47. fun DiceRollerApp() {
48.     var dice1 by remember { mutableStateOf(0) }
49.     var dice2 by remember { mutableStateOf(0) }
50.     val context = LocalContext.current
51.
52.     Column(
53.         modifier = Modifier.fillMaxSize(),
54.         horizontalAlignment = Alignment.CenterHorizontally,
55.         verticalArrangement = Arrangement.Center
56.     ) {
57.         Row(
58.             horizontalArrangement = Arrangement.spacedBy(16.dp)
59.         ) {
60.             DiceImage(diceValue = dice1)
61.             DiceImage(diceValue = dice2)
62.         }
63.
64.         Spacer(modifier = Modifier.height(32.dp))
65.
66.         Button(onClick = {
67.             dice1 = (1..6).random()
68.             dice2 = (1..6).random()
69.
70.             if (dice1 == dice2) {
71.                 Toast.makeText(context, "kongrets dadu
72. dobel", Toast.LENGTH_SHORT).show()
73.             } else {
74.                 Toast.makeText(context, "tidak sama ",
75. Toast.LENGTH_SHORT).show()

```

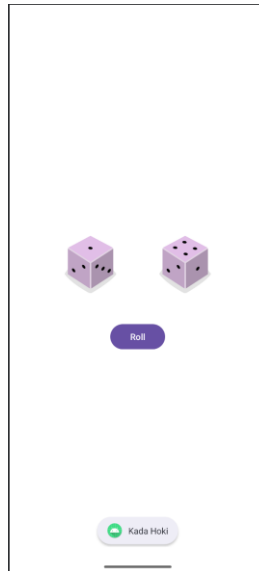
```

74.         }
75.     }) {
76.         Text("Roll")
77.     }
78. }
79. }
80.
81. @Composable
82. fun DiceImage(diceValue: Int) {
83.     val imageResource = when (diceValue) {
84.         1 -> R.drawable.dice_1
85.         2 -> R.drawable.dice_2
86.         3 -> R.drawable.dice_3
87.         4 -> R.drawable.dice_4
88.         5 -> R.drawable.dice_5
89.         6 -> R.drawable.dice_6
90.         else -> R.drawable.dice_0
91.     }
92.
93.     Image(
94.         painter = painterResource(id = imageResource),
95.         contentDescription = "Dadu $diceValue",
96.         modifier = Modifier.size(120.dp)
97.     )
98. }

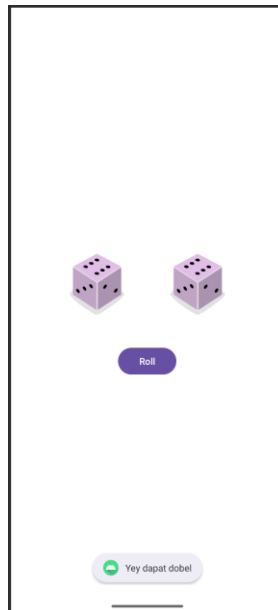
```

B. Output Program

Output XML

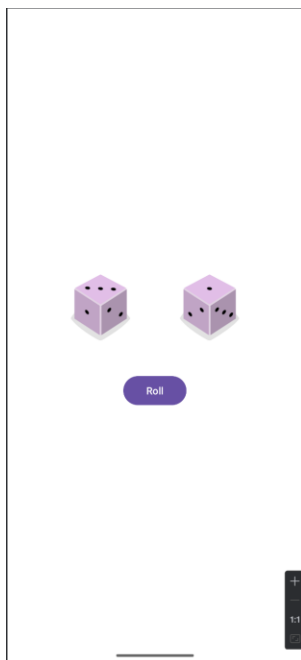


Gambar 1. Screenshot Hasil Jawaban Soal 1 XML

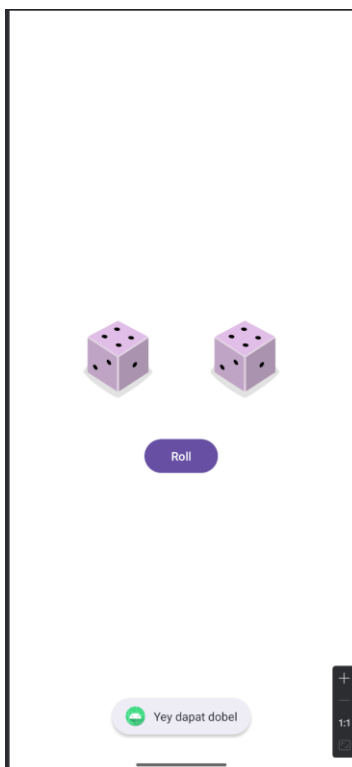


Gambar 2. Screenshot Hasil Jawaban Soal 1 XML

Output Compose



Gambar 3. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose

C. Pembahasan

Pembahasan XML

MainActivity.kt:

Line 1: Merupakan deklarasi package file Kotlin yaitu `com.example.modul1xml`. Ini menentukan identitas dari project aplikasi versi XML kamu.

Line 2: Baris kosong sebagai pemisah.

Line 3 - 7: Blok import yang memasukkan class dan library komponen UI tradisional. Di sini kita menggunakan `android.widget.Button` dan `ImageView` alih-alih foundation Compose. Kita juga mengimpor `AppCompatActivity` sebagai kelas dasar activity tradisional.

Line 8: Baris kosong.

Line 9: Mendeklarasikan class `MainActivity` yang mewarisi (`extends`) `AppCompatActivity()`. Ini adalah pendekatan standar Android sebelum adanya Jetpack Compose.

Line 10: Baris kosong.

Line 11: Meng-override fungsi `onCreate`. Ini adalah titik awal (entry point) di mana siklus hidup aplikasi berjalan saat pertama kali dibuka.

Line 12: Memanggil `super.onCreate` agar sistem Android memproses pembuatan activity bawaannya.

Line 13: `setContentView(R.layout.activity_main)` adalah baris paling krusial di pendekatan XML. Fungsi inilah yang menyambungkan file Kotlin ini dengan file desain `activity_main.xml` yang baru saja kita bahas sebelumnya.

Line 14: Baris kosong.

Line 15: Mendeklarasikan variabel `ivDice1` dan menggunakan fungsi `findViewById<ImageView>` untuk mencari komponen gambar di file XML yang memiliki ID `ivDice1`.

Line 16: Mencari komponen gambar dengan ID `ivDice2` dan menyambungkannya ke variabel `ivDice2`.

Line 17: Mencari komponen tombol dengan ID btnRoll dan menyambungkannya ke variabel btnRoll.

Line 18: Baris kosong.

Line 19: btnRoll.setOnClickListener { memasang listener atau pendeteksi ketukan pada tombol. Semua kode di dalam kurung kurawal { ... } ini hanya akan dieksekusi saat tombol dipencet.

Line 20: Menghasilkan angka acak dari 1 sampai 6, lalu menyimpannya di variabel dice1.

Line 21: Menghasilkan angka acak dari 1 sampai 6, lalu menyimpannya di variabel dice2.

Line 22: Baris kosong.

Line 23: Memanggil fungsi setImageResource pada ivDice1 untuk mengganti gambar yang sedang tampil. Gambar barunya didapat dengan memanggil fungsi getDiceImage dan memberikan nilai dari dice1.

Line 24: Melakukan hal yang sama untuk mengubah gambar pada ivDice2 berdasarkan nilai dari dice2.

Line 25: Baris kosong.

Line 26: Membuka kondisional if untuk membandingkan apakah nilai dice1 sama dengan dice2.

Line 27: Jika kembar, aplikasi memunculkan pesan Toast berbunyi "Yey dapat dobel".

Line 28: Blok else yang dieksekusi jika nilai dadunya berbeda.

Line 29: Memunculkan pesan Toast berbunyi "Kada Hoki".

Line 30 - 32: Penutup kurung kurawal untuk blok if-else, blok setOnClickListener, dan blok fungsi onCreate.

Line 33: Baris kosong.

Line 34: Mendeklarasikan fungsi tambahan bernama `getDiceImage` yang bersifat `private` (hanya bisa diakses di file ini). Fungsi ini meminta input berupa `Int` (angka dadu) dan akan menghasilkan output `Int` (ID unik dari resource gambar di sistem Android).

Line 35: Membuka fungsi `return` dengan kondisi `when` (mirip fungsi `switch-case`) untuk mengecek nilai dadu yang masuk.

Line 36 - 41: Memetakan angka 1 sampai 6 ke file gambar yang sesuai di folder `drawable`. Sistem menganggap `R.drawable.nama_file` sebagai tipe data `Int`.

Line 42: Mengatur kondisi `else` sebagai default atau cadangan jika angka yang masuk tidak sesuai (misal saat tampilan awal), maka kembalikan gambar `dice_0`.

Line 43 - 45: Penutup kurung kurawal untuk blok `when`, penutup fungsi `getDiceImage`, dan penutup kelas `MainActivity`.

activity_main.xml

Line 1: `<?xml version="1.0" encoding="utf-8"?>` mendeklarasikan versi XML dan encoding karakter yang digunakan (UTF-8) agar sistem bisa membaca teks dengan format standar.

Line 2: Membuka tag `<LinearLayout>` utama. Ini bertindak sebagai `root/parent layout` atau wadah paling luar. Atribut `xmlns:android` mendeklarasikan namespace Android yang wajib ada di setiap `root layout`.

Line 3: `android:layout_width="match_parent"` mengatur lebar wadah utama ini agar memenuhi seluruh layar device dari kiri ke kanan.

Line 4: `android:layout_height="match_parent"` mengatur tinggi wadah utama ini agar memenuhi seluruh layar dari atas ke bawah.

Line 5: `android:gravity="center"` menarik semua komponen di dalamnya (dadu dan tombol) tepat ke tengah layar, baik secara vertikal maupun horizontal.

Line 6: `android:orientation="vertical"` mengatur arah susunan komponen dari atas ke bawah. Ini membuat letak tombol otomatis berada di bawah dadu.

Line 7: `android:background="#FFFFFF">` memberikan warna latar belakang putih pada layar, sekaligus menutup awalan tag `LinearLayout` utama.

Line 8: Baris kosong sebagai pemisah.

Line 9: Membuka tag `<LinearLayout>` kedua (child layout). Wadah yang ada di dalam wadah utama ini digunakan khusus untuk menjejerkan kedua gambar dadu.

Line 10: `android:layout_width="wrap_content"` mengatur lebar layout kedua ini agar ukurannya pas mengikuti isi kontennya (dua dadu).

Line 11: `android:layout_height="wrap_content"` mengatur tinggi layout kedua ini agar pas mengikuti tinggi kontennya.

Line 12: `android:orientation="horizontal"` mengatur arah susunan di dalam layout ini dari kiri ke kanan. Inilah yang membuat Dadu 1 dan Dadu 2 letaknya bersebelahan, bukan bertumpuk.

Line 13: `android:layout_marginBottom="32dp">` memberikan jarak (margin) sebesar 32dp di bagian bawah wadah ini, agar dadu tidak terlalu menempel dengan tombol "Roll", sekaligus menutup awalan tag.

Line 14: Baris kosong.

Line 15: Membuka tag komponen `<ImageView>` untuk menampilkan gambar dadu pertama.

Line 16: `android:id="@+id/ivDice1"` mendaftarkan ID unik bernama `ivDice1`. ID ini sangat penting karena Kotlin menggunakan ID ini untuk mencari dan mengubah gambar dadu 1 nantinya.

Line 17: `android:layout_width="120dp"` mengatur agar lebar gambar dikunci menjadi 120dp.

Line 18: `android:layout_height="120dp"` mengatur agar tinggi gambar dikunci menjadi 120dp.

Line 19: `android:layout_marginEnd="16dp"` memberikan ruang kosong sebesar 16dp di sisi kanan dadu 1, agar tidak saling menempel dengan dadu 2.

Line 20: `android:src="@drawable/dice_0"` mengatur gambar default yang ditampilkan pertama kali, yaitu file `dice_0` dari folder `drawable`.

Line 21: `android:contentDescription="Dadu 1" />` memberikan teks deskripsi gambar untuk membantu fitur pembaca layar (aksesibilitas), sekaligus menandai penutup dari tag `<ImageView>` pertama.

Line 22: Baris kosong.

Line 23: Membuka tag komponen `<ImageView>` untuk menampilkan gambar dadu kedua.

Line 24: `android:id="@+id/ivDice2"` mendaftarkan ID unik `ivDice2`.

Line 25: `android:layout_width="120dp"` mengatur lebar gambar dadu 2 menjadi 120dp.

Line 26: `android:layout_height="120dp"` mengatur tinggi gambar dadu 2 menjadi 120dp.

Line 27: `android:layout_marginStart="16dp"` memberikan ruang kosong 16dp di sisi kiri dadu 2, untuk mengimbangi margin dari dadu 1 tadi.

Line 28: `android:src="@drawable/dice_0"` mengatur gambar default awal ke `dice_0`.

Line 29: `android:contentDescription="Dadu 2" />` deskripsi aksesibilitas untuk dadu kedua dan menutup tag `<ImageView>` kedua.

Line 30: `</LinearLayout>` adalah penutup tag dari wadah dadu (child layout horizontal). Semua komponen di luar baris ini tidak akan diletakkan bersebelahan lagi.

Line 31: Baris kosong.

Line 32: Membuka tag komponen `<Button>` untuk membuat tombol.

Line 33: `android:id="@+id/btnRoll"` memberikan ID unik `btnRoll`. Kotlin akan menggunakan ID ini untuk mendeteksi kapan pengguna mengeklik tombolnya.

Line 34: `android:layout_width="wrap_content"` membuat lebar tombol mengikuti seberapa panjang tulisan teks di dalamnya.

Line 35: `android:layout_height="wrap_content"` membuat tinggi tombol mengikuti ukuran teks.

Line 36: `android:text="Roll" />` mengatur tulisan yang muncul di dalam tombol tersebut menjadi "Roll", sekaligus menutup tag `<Button>`.

Line 37: Baris kosong.

Line 38: `</LinearLayout>` menutup tag root layout utama dari keseluruhan halaman. Ini menandakan akhir dari rancangan UI di halaman aplikasi tersebut.

Pembahasan Compose

Line 1: Merupakan deklarasi package file Kotlin yaitu `com.example.modul1compose`. Package ini menentukan identitas unik dari aplikasi dalam ekosistem Android.

Line 2: Baris kosong sebagai pemisah antara deklarasi package dan blok import.

Line 3 - 28: Mengimpor class, library, dan komponen UI dari Jetpack Compose yang dibutuhkan oleh aplikasi:

- `android.os.Bundle`: Digunakan saat activity dibuat untuk menyimpan state (keadaan) dari aplikasi.
- `android.widget.Toast`: Digunakan untuk menampilkan pesan notifikasi singkat (pop-up) di layar.
- `androidx.activity.ComponentActivity`: Kelas dasar dari activity yang secara khusus dirancang untuk mendukung framework Jetpack Compose.
- `androidx.activity.compose.setContent`: Fungsi untuk menentukan UI Compose mana yang akan di-render dan ditampilkan ke layar HP.
- Komponen Foundation & Layout (`Image`, `Column`, `Row`, `Spacer`, `Arrangement`, `Alignment`): Digunakan untuk mengatur tata letak dan posisi elemen, serta menampilkan gambar.
- Komponen Material3 (`Button`, `MaterialTheme`, `Surface`, `Text`): Elemen UI modern yang mengikuti standar sistem desain Material You dari Google.
- Komponen Runtime (`Composable`, `mutableStateOf`, `remember`): Digunakan untuk mendeklarasikan fungsi UI buatanmu dan mengelola state yang dapat memicu perubahan tampilan otomatis.
- Komponen UI tambahan (`LocalContext`, `painterResource`, `dp`): Digunakan untuk mengambil resource konteks aplikasi, file gambar di folder `drawable`, dan mendefinisikan satuan ukuran visual.

Line 29: Baris kosong sebagai pemisah.

Line 30: Mendeklarasikan class MainActivity yang merupakan turunan (subclass) dari ComponentActivity(). Ini adalah entry point (titik awal) jalannya aplikasi ketika dibuka.

Line 31: Meng-override fungsi onCreate. Fungsi ini dipanggil pertama kali oleh sistem Android saat activity ini diciptakan. Parameter savedInstanceState digunakan untuk memulihkan state jika aplikasi ditutup lalu dibuka lagi.

Line 32: Memanggil fungsi onCreate dari superclass untuk memastikan proses inisialisasi sistem Android bawaan berjalan normal.

Line 33: Memanggil blok setContent, tempat di mana kamu mulai "menggambar" tampilan UI menggunakan Jetpack Compose.

Line 34: Menerapkan MaterialTheme, yang memberikan tema visual bawaan (seperti warna dasar dan jenis huruf).

Line 35 - 38: Membuat komponen Surface yang berfungsi sebagai "kanvas" atau latar belakang. Modifier.fillMaxSize() membuatnya membentang memenuhi seluruh layar, dan diberi warna background sesuai tema Material.

Line 39: Memanggil fungsi Composable DiceRollerApp(), yang berisi logika utama dan susunan tampilan aplikasimu.

Line 40 - 44: Penutup blok kurung kurawal untuk Surface, MaterialTheme, setContent, fungsi onCreate, dan class MainActivity.

Line 45: Baris kosong sebagai pemisah.

Line 46: Anotasi @Composable yang memberi tahu compiler bahwa fungsi di bawahnya adalah komponen UI Jetpack Compose.

Line 47: Mendeklarasikan fungsi DiceRollerApp() yang memuat tata letak tombol dan dadu.

Line 48: Menginisialisasi variabel state dice1 menggunakan remember dan mutableStateOf(0). Ini akan menyimpan nilai dadu pertama (dimulai dari angka 0) dan akan memicu recomposition (pembaruan layar) secara otomatis jika nilainya berubah nanti.

Line 49: Menginisialisasi variabel state `dice2` untuk dadu kedua, cara kerjanya persis sama seperti `dice1`.

Line 50: Mengambil Context dari environment Android saat ini dan menyimpannya di variabel `context`. Context ini wajib dimiliki jika kita ingin memunculkan Toast.

Line 51: Baris kosong.

Line 52 - 56: Membuat layout Column (menyusun elemen dari atas ke bawah). `modifier = Modifier.fillMaxSize()` membuatnya sepenuh layar. Atribut `alignment` dan `arrangement` memastikan isinya berada persis di tengah-tengah layar baik secara vertikal maupun horizontal.

Line 57 - 59: Membuat layout Row (menyusun elemen dari kiri ke kanan) di dalam Column tadi untuk menjejerkan dua dadu. `Arrangement.spacedBy(16.dp)` memberikan jarak kosong/spasi sebesar 16dp di antara kedua dadu tersebut.

Line 60: Memanggil fungsi `DiceImage` untuk menampilkan gambar dadu pertama, dengan parameter nilai dari variabel `dice1`.

Line 61: Memanggil fungsi `DiceImage` untuk menampilkan gambar dadu kedua, dengan parameter nilai dari variabel `dice2`.

Line 62: Penutup blok Row.

Line 63: Baris kosong.

Line 64: Menambahkan komponen `Spacer` setinggi 32.dp. Ini berfungsi untuk memberikan jarak/margin bawah antara gambar dadu dan tombol "Roll" di bawahnya.

Line 65: Baris kosong.

Line 66: Membuat komponen `Button` (Tombol). Blok { setelah kata `onClick` adalah tempat di mana kita menaruh aksi saat tombol tersebut ditekan.

Line 67: Mengubah nilai state `dice1` menjadi angka acak (menggunakan `.random()`) dalam rentang 1 sampai 6.

Line 68: Mengubah nilai state `dice2` menjadi angka acak dari 1 sampai 6.

Line 69: Baris kosong.

Line 70: Membuka kondisional if untuk mengecek apakah angka yang didapat dadu 1 sama dengan angka dadu 2.

Line 71: Jika kondisinya benar (angkanya sama/dobel), maka aplikasi memanggil Toast.makeText untuk memunculkan pesan "kongrets dadu dobel".

Line 72: Blok else yang akan dieksekusi jika angka kedua dadu berbeda.

Line 73: Memunculkan pesan Toast dengan teks "tidak sama ".

Line 74: Penutup blok if-else.

Line 75: Penutup blok onClick pada Button. Blok kurung kurawal berikutnya mendefinisikan isi dari tombol tersebut.

Line 76: Menambahkan komponen Text bertuliskan "Roll" ke dalam tombol.

Line 77 - 79: Penutup blok Button, Column, dan fungsi DiceRollerApp().

Line 80: Baris kosong.

Line 81: Anotasi @Composable untuk menandai fungsi pembuatan gambar dadu di bawahnya.

Line 82: Mendeklarasikan fungsi DiceImage yang menerima satu parameter bernama diceValue dengan tipe data Integer (angka bulat).

Line 83: Mendeklarasikan konstanta imageResource yang isinya ditentukan oleh pengkondisian when (mirip switch-case) berdasarkan parameter diceValue.

Line 84 - 89: Memetakan angka 1 sampai 6 agar menggunakan gambar yang sesuai dari folder drawable (misalnya: jika nilainya 1, gunakan R.drawable.dice_1).

Line 90: Menangani kondisi else (misalnya saat aplikasi pertama dijalankan dan state masih bernilai awal 0), maka yang ditampilkan adalah gambar R.drawable.dice_0.

Line 91: Penutup blok when.

Line 92: Baris kosong.

Line 93: Membuat komponen UI Image untuk menampilkan gambar ke layar.

Line 94: Mengisi atribut painter dengan painterResource dari variabel imageResource yang sudah ditentukan oleh kondisi when tadi.

Line 95: Menambahkan contentDescription sebagai teks bantuan untuk aksesibilitas sistem pembaca layar (Screen Reader).

Line 96: Menerapkan Modifier untuk memaksa ukuran lebar dan tinggi gambar (size) menjadi 120.dp secara konstan/statis.

Line 97 - 98: Penutup blok komponen Image dan fungsi DiceImage

SOAL 2

Riset dan cite minimal 2 buah artikel/jurnal/buku mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatlah opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan.

A. Pembahasan

1. Pendekatan Dasar Penulisan Kode

- Imperatif: Berfokus pada penjabaran algoritma secara terperinci. Pengembang diwajibkan untuk menulis instruksi langkah demi langkah (*step-by-step*) secara manual guna mengubah kondisi (*state*) dari sebuah program.
- Deklaratif: Berfokus pada hasil akhir yang diharapkan, tanpa perlu menyusun instruksi teknis mengenai bagaimana sistem harus mencapainya. Pendekatan ini membuat kode menjadi lebih ringkas, modular, dan mudah dipelihara.

2. Pembuatan Tampilan (UI) Android

- Imperatif (XML dan Kotlin): Merupakan metode konvensional yang membutuhkan penggabungan antara XML dan Kotlin/Java untuk merancang satu tampilan antarmuka. Pengembang harus mencari komponen secara manual (misalnya menggunakan `findViewById`) dan memberikan perintah satu per satu untuk memperbarui tampilan setiap kali terjadi perubahan data.
- Deklaratif (Jetpack Compose): Merupakan teknologi antarmuka modern yang dikembangkan untuk mengatasi kompleksitas XML. Pengembang hanya perlu merancang struktur antarmukanya melalui fungsi komponen, dan sistem akan secara otomatis memperbarui tampilan menyesuaikan dengan data terbaru.

3. Cara Kerja Saat Mendesain Visual

- Imperatif: Menuntut pembaruan tampilan secara manual. Fokus pengembang sering kali terbagi karena desain visual dan logika pembaruan ditulis pada berkas yang berbeda. Pemisahan arsitektur ini membuat struktur kode menjadi panjang dan rentan memicu kesalahan (*bug*), terutama pada aplikasi dengan fitur yang kompleks.
- Deklaratif: Alur kerjanya lebih efisien dan memiliki kemiripan dengan proses perancangan desain visual pada perangkat lunak *prototyping* seperti Figma.

Pengembang hanya perlu menentukan komponen beserta tata letaknya, kemudian sistem akan merender tampilan tersebut tanpa memerlukan instruksi modifikasi yang panjang.

4. Kerapian Kode dan Kerja Tim

- Imperatif: Akibat proses yang terpisah dan dikelola secara manual, pendekatan ini memiliki risiko tinggi terhadap asinkronisasi antara tampilan antarmuka di layar dengan data logika yang sebenarnya.
- Deklaratif: Tampilan layar dapat menyesuaikan bentuk dan isinya secara otomatis. Hal ini menghasilkan barisan kode yang jauh lebih bersih, rapi, dan mudah dibaca saat proses evaluasi sejawat (*peer review*). Kondisi ini memungkinkan pengembang untuk lebih fokus merancang pengalaman pengguna tanpa terbebani oleh kerumitan teknis antarmuka.

5. Preferensi Paradigma

- Berdasarkan perbandingan yang telah dijabarkan, paradigma yang lebih direkomendasikan dan dipilih untuk pengembangan antarmuka adalah *Declarative Programming* (Jetpack Compose).
- Alasan utamanya adalah alur kerja deklaratif sangat relevan dan efisien untuk kebutuhan pengembangan aplikasi modern. Pengembang tidak perlu lagi menghabiskan waktu menulis instruksi teknis yang panjang hanya untuk memodifikasi elemen sederhana saat terjadi interaksi pengguna.
- Struktur kode yang lebih ringkas dan rapi sangat mempermudah proses identifikasi kesalahan (*debugging*). Selain itu, kolaborasi dalam pengerjaan kelompok menjadi lebih efektif karena alur logika kode yang jelas dan terstruktur dengan baik.
- Kesimpulannya, paradigma deklaratif memberikan keleluasaan bagi pengembang untuk memusatkan perhatian pada perancangan fitur dan pengalaman pengguna yang optimal, alih-alih menghabiskan sumber daya pada pengelolaan kode antarmuka yang rumit.

SOAL 3

Apa yang membedakan OOP pada kotlin dengan OOP pada java?

A. Pembahasan

1. Aturan Turunan (Inheritance)

- Java: Secara bawaan, sebuah *class* itu ibarat pintu terbuka; bebas diturunkan atau diwariskan ke *class* lain. Kalau mau dilarang, kamu harus repot-repot menggemboknya pakai kata kunci *final*.
- Kotlin: Pendekatannya dibalik demi keamanan. Semua *class* otomatis digembok (tidak bisa diturunkan). Kalau kamu memang berniat menjadikan *class* tersebut sebagai induk, kamu harus membuka kuncinya dengan menambahkan kata *open*.

2. Urusan Getter dan Setter (Enkapsulasi)

- Java: Untuk membuat data aman, kita wajib mengubah variabel menjadi *private*, lalu mengetik fungsi *getter* dan *setter* secara manual satu per satu. Ujung-ujungnya, file kode jadi penuh dengan puluhan baris yang fungsinya cuma buat mengambil atau mengubah data.
- Kotlin: Sangat praktis karena punya fitur *Properties*. Sistem akan otomatis membuatkan *getter* dan *setter* di belakang layar. Kamu cukup pakai kata *val* jika datanya hanya boleh dibaca (read-only), atau *var* jika datanya boleh dibaca dan diubah. Kodenya pun jadi sangat pendek.

3. Membuat Class Penyimpan Data

- Java: Saat membuat *class* yang murni hanya untuk menampung data (sering disebut POJO), kita harus mengetik banyak fungsi tambahan secara manual seperti konstruktor, *equals()*, *hashCode()*, dan *toString()*.
- Kotlin: Kamu hanya perlu menambahkan kata *data* di depan *class* (menjadi *data class*). Sistem akan otomatis membuatkan semua fungsi tambahan tadi. Selain menghemat waktu, ini juga mengurangi risiko *error* karena salah ketik.

4. Cara Membuat Konstruktor

- Java: Konstruktor dibuat persis seperti fungsi di dalam *class*, dan penamaannya wajib sama persis dengan nama *class* tersebut.
- Kotlin: Lebih rapi dan terstruktur karena dibagi dua. Ada *Primary Constructor* yang bisa langsung diselipkan di sebelah nama *class* pada bagian paling atas (*header*). Jika butuh inisialisasi tambahan yang beda parameter, barulah kita memakai *Secondary Constructor* di dalam tubuh *class*.

5. Pengganti Fitur Static

- Java: Sangat bergantung pada kata kunci *static* untuk membuat fungsi atau variabel yang menempel langsung pada *class*, bukan pada cetakan objeknya.
- Kotlin: Fitur *static* dihapus total agar konsep OOP-nya lebih murni. Sebagai gantinya, Kotlin menyediakan *companion object*. Secara fungsi, penggunaannya mirip dengan *static* di mana kita bisa memanggil fungsinya secara langsung, namun di dalam memori ia tetap hidup sebagai objek tunggal (*singleton*) yang nyata.

.

DAFTAR PUSTAKA

- Kusuma, M., & Rifani, A. H. (2023). Comparison analysis of Jetpack Compose and Flutter in Android-based application development using Technical Domain. *2023 Eighth International Conference on Informatics and Computing (ICIC)*.
- Simarmata, S., & Karo karo, P. (2026). Comparative Evaluation of Functional, Object-Oriented, and Declarative Programming Paradigms for Scalability and Maintainability in Distributed Data Processing Applications. *Programming and Algorithm Fundamentals*, 1(1), 29-37.

TAUTAN GIT

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/ZulFath1/PraktikumMobile.git>